Database project: FitTrackDB

Prepared by:
Rama Alhuthali | 446001814
Shaimaa Alzahrani |446001735
Sama AlSilaimi |446004294

Course name: introduction to database
Instructor name: sumayah bayounis

Table of Contents

Work organization ---------- 3

Introduction ---------- 4

Conceptual and logical design documentation ---------- 4-6

ER diagram ---------- 7

Relational schema ---------- 8

Additional use case ---------- 8

Implementation ---------- 9-14

Work organization

Name	Works on
Rama alhuthali	(ERD, additional use case, query implementation, writing document)
Shaimaa Hussein Alzahrani	(query implementation, and writing its explanation)
Sama Eidhah AlSilaimi	(query implementation, Relational Sckema)

Introduction

The **FitTrackDB** is a fitness and nutrition tracking system designed to help users manage their physical health by monitoring dietary intake and physical exertion.

Conceptual & logical design documentation

1. Conceptual:

1.1. Entities

User

Represents individuals using the system.

- Attributes: user_id, first_name, last_name, age, gender, weight, height

Food

Represents food items consumed by users.

- Attributes: food_id, name, calories

Activity

Represents physical activities performed by users.

- Attributes: activity_id, name, calories_burned

1.2. Relationships

User–Food Relationship

- Type: Many-to-Many (M:N)
- Description: A user can consume multiple food items, and each food item can be consumed by multiple users.
- Implementation: user_food table

User–Activity Relationship

- Type: Many-to-Many (M:N)
- Description: A user can perform multiple activities, and each activity can be performed by multiple users.
- Implementation: user_activity table

Business Rules & Logic

- **Identity**: Users are uniquely identified by their full name to prevent duplicate profiles for the same individual.
- **Data Integrity:** Caloric values for both food and activities must be positive integers.

- **Automated Metrics:** The system automatically calculates a user's Basal Metabolic Rate (BMR) and recommended macronutrient distribution (Protein/Carbs/Fats) based on the Mifflin-St Jeor Equation.

2. Logical documentation

2.1. Tables structure

Users Table

- **user_id:** INT, Primary Key, Auto Increment
- **first_name:** VARCHAR(30), NOT NULL
- **last_name:** VARCHAR(30), NOT NULL
- **age:** INT
- **gender**: VARCHAR(10)
- **weight:** DECIMAL(5,2)
- **height:** DECIMAL(5,2)
- **Constraint:** UNIQUE(first_name, last_name)

Food Table

- **food_id**: INT, Primary Key, Auto Increment
- **name:** ج50), NOT NULL, UNIQUE
- **calories:** INT, NOT NULL, CHECK (calories > 0)

Activity Table

- **activity_id:** INT, Primary Key, Auto Increment
- **name:** VARCHAR(50), NOT NULL, UNIQUE
- **calories_burned**: INT, NOT NULL, CHECK (calories_burned > 0)

user_food Table (Junction Table)

- **user_id:** INT**, Foreign Key --** Users(user_id)
- **food_id:** INT**, Foreign Key --** Food(food_id)
- **Primary Key:** (user_id, food_id)

user_activity Table (Junction Table)

- **user_id:** INT**, Foreign Key --** Users(user_id)
- **activity_id:** INT**, Foreign Key --** Activity(activity_id)
- **Primary Key:** (user_id, activity_id)

2.2. Integrity constrains

 Primary Keys: Ensure each record is uniquely identifiable.
 Foreign Keys: Maintain referential integrity between related tables.

Check Constraints:

- calories > 0
- calories_burned > 0

Unique Constraints:

- Food names must be unique.
- Activity names must be unique.
- Combination of first_name and last_name must be unique.

3. Derived data (view)

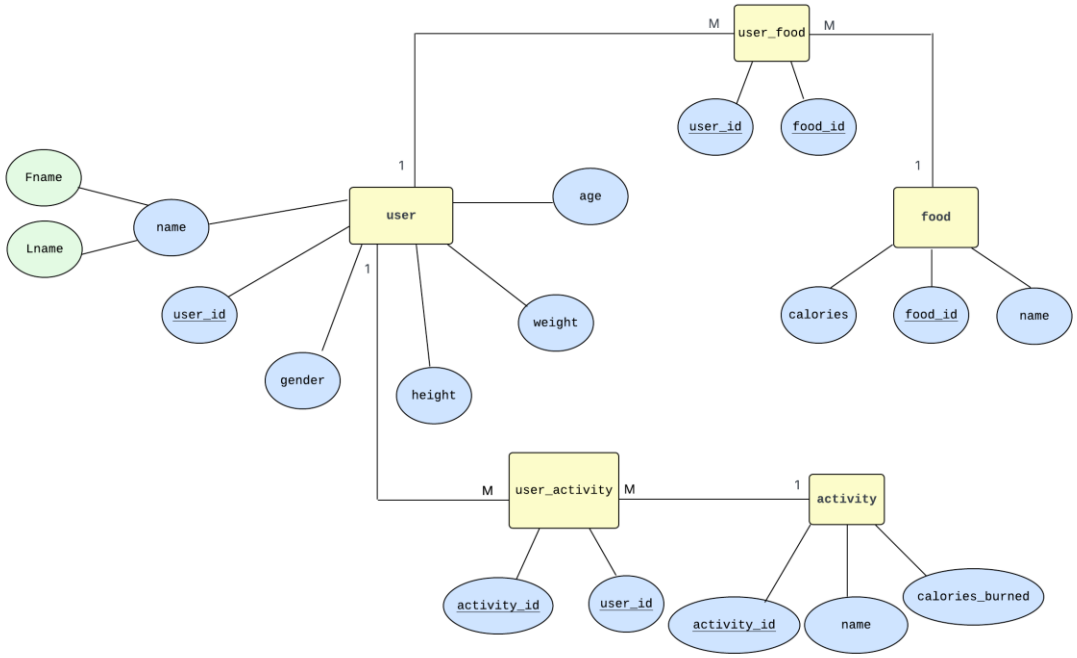
calories_report View

The calories_report view calculates daily calorie requirements and macronutrient distribution for each user based on their personal data.

Calculation Logic

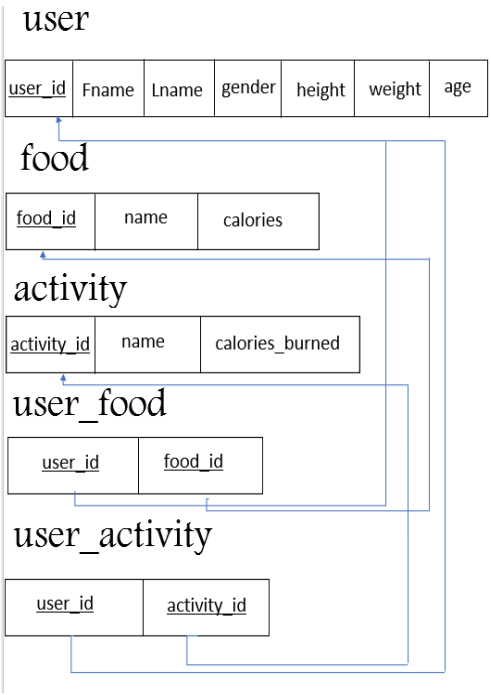
- **For males:**
  - Calories = (10 × weight) + (6.25 × height) − (5 × age) + 5
- **For females:**
  - Calories = (10 × weight) + (6.25 × height) − (5 × age) − 161

ER diagram

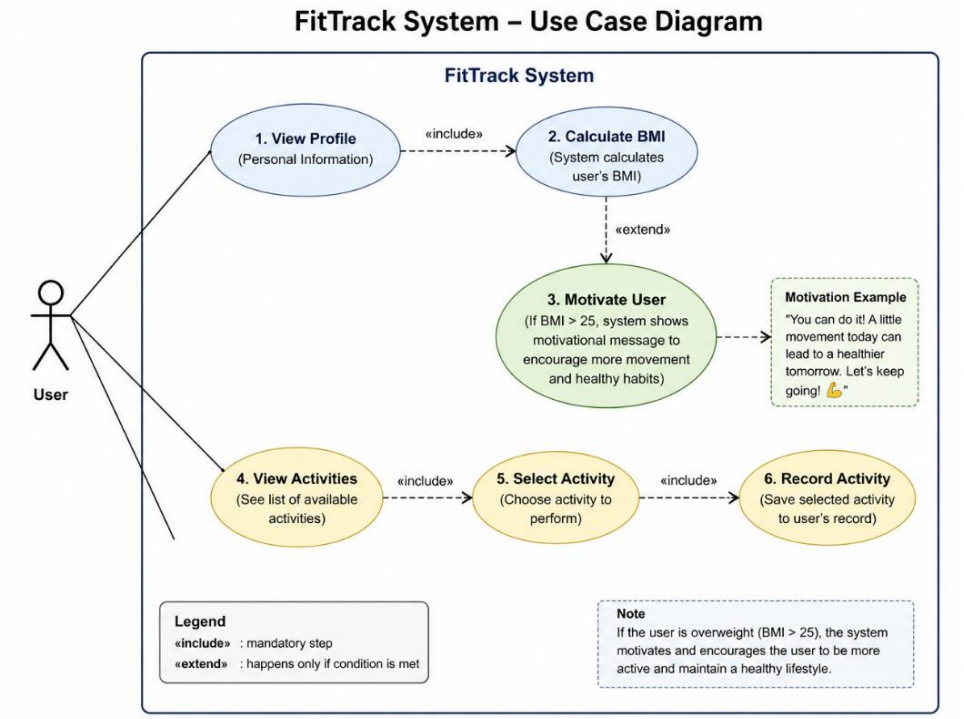


ERD FitTrack

Relation schema



Additional use case – motivation and recommendation system



Implementation

FitTrackDB database is created and selected using USE FitTrackDB.
It contains 3 tables:
1.Users: stores personal data (user_id PK, name, age, gender, weight, height) with UNIQUE constraint on full name.
2.Food: stores food items (food_id PK, name UNIQUE, calories with CHECK > 0).
3.Activity: stores exercises (activity_id PK, name UNIQUE, calories_burned with CHECK > 0).

The design ensures data is organized, consistent, and valid.

```sql
CREATE DATABASE FitTrackDB;
USE FitTrackDB;
CREATE TABLE Users (
    user_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(30) NOT NULL,
    last_name VARCHAR(30) NOT NULL,
    age INT,
    gender VARCHAR(10),
    weight DECIMAL(5,2),
    height DECIMAL(5,2),

    UNIQUE (first_name, last_name));
    CREATE TABLE Food (
    food_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    calories INT NOT NULL CHECK (calories > 0),

    UNIQUE (name)
);
CREATE TABLE Activity (
    activity_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(50) NOT NULL,
    calories_burned INT NOT NULL CHECK (calories_burned > 0),
    UNIQUE (name)
);
```

Two junction tables were created to represent many-to-many relationships.

"user_food": connects users with food items.
Includes:(user_id, food_id).

Composite primary key (user_id, food_id) prevents duplicates**.**

Foreign keys link to Users, Food tables

user_activity :Connects users with activities**.**
Includes: (user_id, activity_id)

Composite primary key (user_id, activity_id) prevents duplicates.

Foreign keys link to Users and Activity tables**.**

```
25    );
26    CREATE TABLE user_food(
27    user_id INT,
28    food_id INT,
29    PRIMARY KEY(user_id,food_id),
30    FOREIGN KEY(user_id)REFERENCES Users(user_id),
31    FOREIGN KEY(food_id)REFERENCES Food(food_id)
32    );
33    CREATE TABLE user_activity(
34    user_id INT,
35    activity_id INT,
36    PRIMARY KEY(user_id,activity_id),
37    FOREIGN KEY(user_id)REFERENCES Users(user_id),
38    FOREIGN KEY(activity_id)REFERENCES Activity(activity_id)
39    );
```

Data Insertion

we inserted sample records into all tables using the INSERT INTO statement.

In "Users"Table:

we added Five users with personal information:(first name, last name, age, gender ,weight,height)

In " Food "Table:

we added Five food items with their calories values: (Rice, Chicken, Apple, Bread, Milk)

In "Activity" Table:

also we added five physical activities:
(Running, Walking, Cycling, Swimming, Gym)

In "user_food" Table:

Here we inserted records to link users with consumed food items.

In "user_activity" Table:

we inserted records to link users with performed activities.

```
Limit to 1000 rows
40  INSERT INTO Users (first_name,last_name, age, gender, weight, height) VALUES
41  ('mohammad','Alzahrani', 22, 'male', 70, 175),
42  ('Sama','almehmadi', 20, 'female', 55, 160),
43  ('Omar','al-saleem', 25, 'male', 80, 180),
44  ('Lama','alharthi', 23, 'female', 60, 165),
45  ('Noor','alfahmi', 21, 'female', 50, 158);
46  INSERT INTO Food (name, calories) VALUES
47  ('Rice', 200),
48  ('Chicken', 250),
49  ('Apple', 80),
50  ('Bread', 150),
51  ('Milk', 120);
52  INSERT INTO Activity (name, calories_burned) VALUES
53  ('Running', 300),
54  ('Walking', 150),
55  ('Cycling', 250),
56  ('Swimming', 350),
57  ('Gym', 400);
58
59  INSERT INTO user_food VALUES
60  (1,4),(2,3),(3,1),(4,5),(5,2);
61  INSERT INTO user_activity VALUES
62  (1,3),(2,1),(3,4),(4,2),(5,5);
63
```

Aggregate Functions

Here we use (aggregate functions) to summarize and analyze the stored data.
first: use **"SUM(calories)"** to calculate the total calories of all food items.
Result : 800
second: use "(weight)" to calculate the average weight of users.
Result: 63
third: we use "COUNT(*)"with "GROUP BY gender" to count users based on gender.
Result:
Male: 2
Female: 3

```
98
99 •    SELECT SUM(calories) AS total_calories
100     FROM Food;
```

Result Grid | Filter Rows: | Export:

total_calories
800

```
93
94 •    SELECT AVG(weight) AS average_weight
95      FROM Users;
96
97
```

Result Grid | Filter Rows: | Export:

average_weight
63.000000

```
88
89 •    SELECT gender, COUNT(*) AS total_users
90      FROM Users
91      GROUP BY gender;
92
```

Result Grid | Filter Rows: | Export:

gender	total_users
male	2
female	3

View (derived information)

The CASE statement is SQL's if-then logic. It calculates the Basal Metabolic Rate (BMR) by choosing a specific formula based on the user's gender
It allows a single column (calories) to hold different calculations for different types of users.

```
64  CREATE VIEW calories_report AS
65  SELECT
66      user_id,
67      first_name,
68      last_name,
69      calories,
70      (calories * 0.30) / 4 AS protein_grams,
71      (calories * 0.40) / 4 AS carbs_grams,
72      (calories * 0.30) / 9 AS fats_grams
73  FROM (
74      SELECT
75          user_id,
76          first_name,
77          last_name,
78          CASE
79              WHEN gender = 'male' THEN
80                  (10 * weight + 6.25 * height - 5 * age + 5)
81              ELSE
82                  (10 * weight + 6.25 * height - 5 * age - 161)
```

```sql
FROM (
    SELECT
        user_id,
        first_name,
        last_name,
        CASE
            WHEN gender = 'male' THEN
                (10 * weight + 6.25 * height - 5 * age + 5)
            ELSE
                (10 * weight + 6.25 * height - 5 * age - 161)
        END AS calories
    FROM Users
) AS sub;

SELECT * FROM  calories_report;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

user_id	first_name	last_name	calories	protein_grams	carbs_grams	fats_grams
1	mohammad	Alzahrani	1688.7500	126.6562500000	168.8750000000	56.2916666667
2	Sama	almehmadi	1289.0000	96.6750000000	128.9000000000	42.9666666667
3	Omar	al-saleem	1805.0000	135.3750000000	180.5000000000	60.1666666667
4	Lama	alharthi	1355.2500	101.6437500000	135.5250000000	45.1750000000
5	Noor	alfahmi	1221.5000	91.6125000000	122.1500000000	40.7166666667

This view provides a personalized calorie and nutrition report for users.